

数学实验与数学软件

第十二课：MATLAB多项式运算 与动态规划

谭军

中山大学数学学院

教师邮箱：mcstj@mail.sysu.edu.cn



MATLAB多项式表示与运算

□ MATLAB习惯将降幂排列的多项式 $a(x) = a_1x^n + a_2x^{n-1} + \dots + a_nx + a_{n+1}$ 存储为系数行向量 $\mathbf{a} = [a_1, a_2, \dots, a_{n+1}]$

□ 多项式的乘法，基于MATLAB的多项式系数定义，有：

$$\sum_{i=1}^{m+1} a_i x^{m+1-i} \cdot \sum_{j=1}^{n+1} b_j x^{n+1-j} \\ = \sum_{k=2}^{m+n+2} \left(\sum_{i=\max(1, k-n-1)}^{\min(m+1, k-1)} a_i b_{k-i} \right) x^{m+n+2-k}$$

注意到红字，即乘积多项式的系数恰为两个原始系数的卷积

□ $\mathbf{c} = \text{conv}(\mathbf{a}, \mathbf{b})$ 就是一种快速获取多项式相乘系数的方法

□ $[\mathbf{q}, \mathbf{r}] = \text{deconv}(\mathbf{b}, \mathbf{a})$ 为多项式带余除法， $\mathbf{q}$ 为商 $\mathbf{r}$ 为余式

□ $[\mathbf{r}, \mathbf{p}, \mathbf{k}] = \text{residue}(\mathbf{b}, \mathbf{a})$ 即表示 $\frac{b(x)}{a(x)} = \sum_{i=1}^n \frac{r_i}{x-p_i} + k(x)$ ，
其中 $\mathbf{r}$ 为留数向量， $\mathbf{p}$ 为极点向量， $\mathbf{k}$ 为余式系数行向量

MATLAB多项式表示与运算

- ❑ **$r = \text{roots}(a)$** 求多项式 $a(x)$ 的所有根（含重根和任意复根）。可以注意到该函数返回的根用列向量存储。
- ❑ **$a = \text{poly}(r)$** 当 r 为一个行向量时， a 为根等于 r 的多项式（即函数 roots 的逆运算）；而当 r 为方阵时， poly 函数将获得 r 的特征多项式（用于计算特征值）
- ❑ **$V = \text{polyval}(p, x)$** p 为一个多项式的系数行向量， x 可以是任意规模的矩阵，此函数将 x 每一个元素代入 $p(x)$ 并获得代入后的数值矩阵 $p(x)$
- ❑ **$V = \text{polyvalm}(p, x)$** 的含义略有不同，此函数意味计算以矩阵 x 为整体的多项式代入的结果矩阵，即 $V = p_1 X^n + p_2 X^{n-1} + \dots + p_n X + p_{n+1} I$

例4.4-1

□ `poly2str(a, 's')` 将以a向量元素为系数，s为自变量生成多项式字符串，按降幂顺序排列。

【例4.4-1】多项式带余除法实例（有改动）

```
format rat                                     %分数表示数值
p1=conv([1,0,2],conv([1,4],[1,1])); %计算分子多项式( $s^2 + 2$ )( $s + 4$ )( $s + 1$ )
p2=[1 0 1 1];                                %分母多项式 $s^3 + 0 \cdot s^2 + s + 1$ 
```

```
[q,r]=deconv(p1,p2);                          %带余除法的计算
cq='商多项式为   ' ; cr='余多项式为   ' ;
disp([cq,poly2str(q,'s')]),disp([cr,poly2str(r,'s')])
```

```
商多项式为      s + 5
余多项式为      5 s^2 + 4 s + 3
```

```
qp2=conv(q,p2);                                %回代pp1=商x分母+余式
pp1=qp2+r;
```

```
pp1==p1    %向量元素逐一比对，每一个相同则有一个1，五个1代表pp1与p1完全相同
           %实际操作中如果系数非整数，不建议使用==来判断两个向量是否完全相同
```

```
ans =
      1      1      1      1      1
```

例4.4-2

【例 4.4-2】矩阵的特征多项式与特征根

```
format short, A=[11 12 13;14 15 16;17 18 19];
PA=poly(A)                                     %求矩阵A的特征多项式
PA = 1.0000   -45.0000   -18.0000    0.0000
PPA=poly2str(PA,'s')                           %字符串式显示多项式
PPA = s^3 - 45 s^2 - 18 s + 1.6206e-014

s=eig(A)
s = 45.3965
    -0.3965
         0.0000

r=roots(PA);                                   %特征多项式的根与特征值相同 (r=s)

n = length(PA);                               %多项式次数+1
AA = diag(ones(1,n-2,class(PA)),-1);
%用长度为n-2的向量生成一个向下平移一行的(n-1)x(n-1)维矩阵
AA
AA =
```

0	0	0
1	0	0
0	1	0

例4.4-2,3

```
AA(1,:) = -PA(2:n) ./ PA(1);AA %第一行减掉了特征多项式后三个系数
```

```
AA =
```

```
45.0000    18.0000   -0.0000
 1.0000         0         0
         0    1.0000         0
```

```
sr = eig(AA);sr' %该矩阵AA的特征多项式与PA相同，特征值也相同
```

```
ans =    45.3965   -0.3965    0.0000
```

【例 4.4-3】构造特征根的多项式

```
R=[-0.5,-0.3+0.4*i,-0.3-0.4*i]; %三个特征根，复根需有共轭复数对
```

```
P=poly(R) %生成对应的多项式
```

```
P =
```

```
1.0000    1.1000    0.5500    0.1250
```

```
PR=real(P) %取P的实部
```

```
PR =
```

```
1.0000    1.1000    0.5500    0.1250
```

```
PPR=poly2str(PR,'x') %转为多项式字符串
```

```
PPR =
```

```
x^3 + 1.1 x^2 + 0.55 x + 0.125
```

例4.4-4

【例 4.4-4】polyval与polyvalm函数

```
clear
```

```
p=[1,2,3];
```

```
poly2str(p,'x')
```

%定义多项式 $p(x)$

```
ans =
```

```
    x^2 + 2 x + 3
```

```
X=[1,2;3,4]
```

%定义2x2矩阵 X

```
X =
```

```
     1     2  
     3     4
```

```
va=X.^2+2*X+3
```

%对应元素代入多项式 $p(x)$

```
va =
```

```
     6    11  
    18    27
```

```
Va=polyval(p,X)
```

%等价的MATLAB函数polyval

```
Va =
```

```
     6    11  
    18    27
```

例4.4-4

```
vm=X^2+2*X+3*eye(2)
```

%将矩阵整体代入 $p(x)$

```
vm =
```

```
    12    14  
    21    33
```

```
Vm=polyvalm(p,X)
```

%等价的MATLAB函数

```
Vm =
```

```
    12    14  
    21    33
```

```
cp=poly(X);poly2str(cp,'x')
```

%获取并显示 x 的特征多项式

```
ans =
```

```
    x^2 - 5 x - 2
```

```
cpXa=polyval(cp,X)
```

%代入 x 的元素，得到结果矩阵

```
cpXa =
```

```
    -6    -8  
    -8    -6
```

```
cpX=polyvalm(cp,X)
```

%整体代入验证H-C定理

```
cpX =
```

```
1.0e-015 *  
    0.2220         0  
         0    0.2220
```


MATLAB有限长向量卷积函数conv

- 已知 $a = [a_1, a_2, \dots, a_n]$, $b = [b_1, b_2, \dots, b_m]$, 类似于刚才多项式乘法的例子, **conv(a,b)** 函数将返回 $a * b$ 的向量, 即

$$(a * b)_k = \sum_{i=1}^{k-1} a_i b_{k-i}, 2 \leq k \leq m + n,$$

这里补充定义 $\forall i > n, a_i = 0, \forall j > m, b_j = 0$

- conv(a,b,'same')** 将返回一个长度等于 n 的卷积结果, 即截取上述卷积结果中间 n 个元素的结果, 常用于信号或图像处理问题。若3个选2个, 则默认选择后两个。
- conv(a,b,'valid')** 将返回一个长度等于 $n - m + 1$ 的卷积结果, 即仅包含 b 的全部元素参与卷积的中间项。
- deconv** 为反卷积函数。卷积运算之前对于边界条件的处理有时需要跟实际问题相结合。具体方法有0填充 (默认), 对称延拓, 复制延拓, 周期延拓等。

例4.4-7

【例 4.4-7】有序列 $A(n) = \begin{cases} 1 & n = 3, 4, \dots, 12 \\ 0 & \text{else} \end{cases}$ 和 $B(n) = \begin{cases} 1 & n = 2, 3, \dots, 9 \\ 0 & \text{else} \end{cases}$ ，求这两个序列的卷积。

```
N1=3;N2=12;A=ones(1,(N2-N1+1)); %录入两个长度为10和8的全1向量
M1=2;M2=9;B=ones(1,(M2-M1+1));
Nc1=N1+M1;Nc2=N2+M2;          kcc=Nc1:Nc2; %结果序列的取值范围
for n=Nc1:Nc2
    w=0;
    for k=N1:N2                %利用两层for逐一计算，效率差
        kk=k-N1+1;
        t=n-k;
        if t>=M1&t<=M2
            tt=t-M1+1;
            w=w+A(kk)*B(tt);
        end
    end
    nn=n-Nc1+1;
    cc(nn)=w;                  %
end
kcc,cc
kcc = 5~21 ,cc= 1 2 3 4 5 6 7 8 8 8 7 6 5 4 3 2 1 (示意结果)
```

例4.4-7

【例 4.4-7】有序列 $A(n) = \begin{cases} 1 & n = 3, 4, \dots, 12 \\ 0 & \text{else} \end{cases}$ 和 $B(n) = \begin{cases} 1 & n = 2, 3, \dots, 9 \\ 0 & \text{else} \end{cases}$ ，求这两个序列的卷积。

N1=3;N2=12;a=ones(1,N2+1);a(1:N1)=0;%相同的向量定义，用0填充左侧
M1=2;M2=9;b=ones(1,M2+1);b(1:M1)=0;

c=conv(a,b); %直接conv函数计算

kc=0:(N2+M2);

kc,c %结果与刚才相比左侧填充了5个0

kc =

Columns 1 through 12

0 1 2 3 4 5 6 7 8 9

10 11

Columns 13 through 22

12 13 14 15 16 17 18 19 20 21

c =

Columns 1 through 12

0 0 0 0 0 1 2 3 4 5

6 7

Columns 13 through 22

8 8 8 7 6 5 4 3 2 1

例4.4-7

【例 4.4-7】有序列 $A(n) = \begin{cases} 1 & n = 3, 4, \dots, 12 \\ 0 & \text{else} \end{cases}$ 和 $B(n) = \begin{cases} 1 & n = 2, 3, \dots, 9 \\ 0 & \text{else} \end{cases}$ ，求这两个序列的卷积。

N1=3;N2=12;M1=2;M2=9;

A=ones(1,(N2-N1+1));

B=ones(1,(M2-M1+1));

%去掉左边

的0，与前法类似

C=conv(A,B);

Nc1=N1+M1;Nc2=N2+M2;

%左右端点下标计算

KC=Nc1:Nc2;

KC,C

KC =

Columns 1 through 12

5 6 7 8 9 10 11 12 13 14

15 16

Columns 13 through 17

17 18 19 20 21

C =

Columns 1 through 12

1 2 3 4 5 6 7 8 8 8

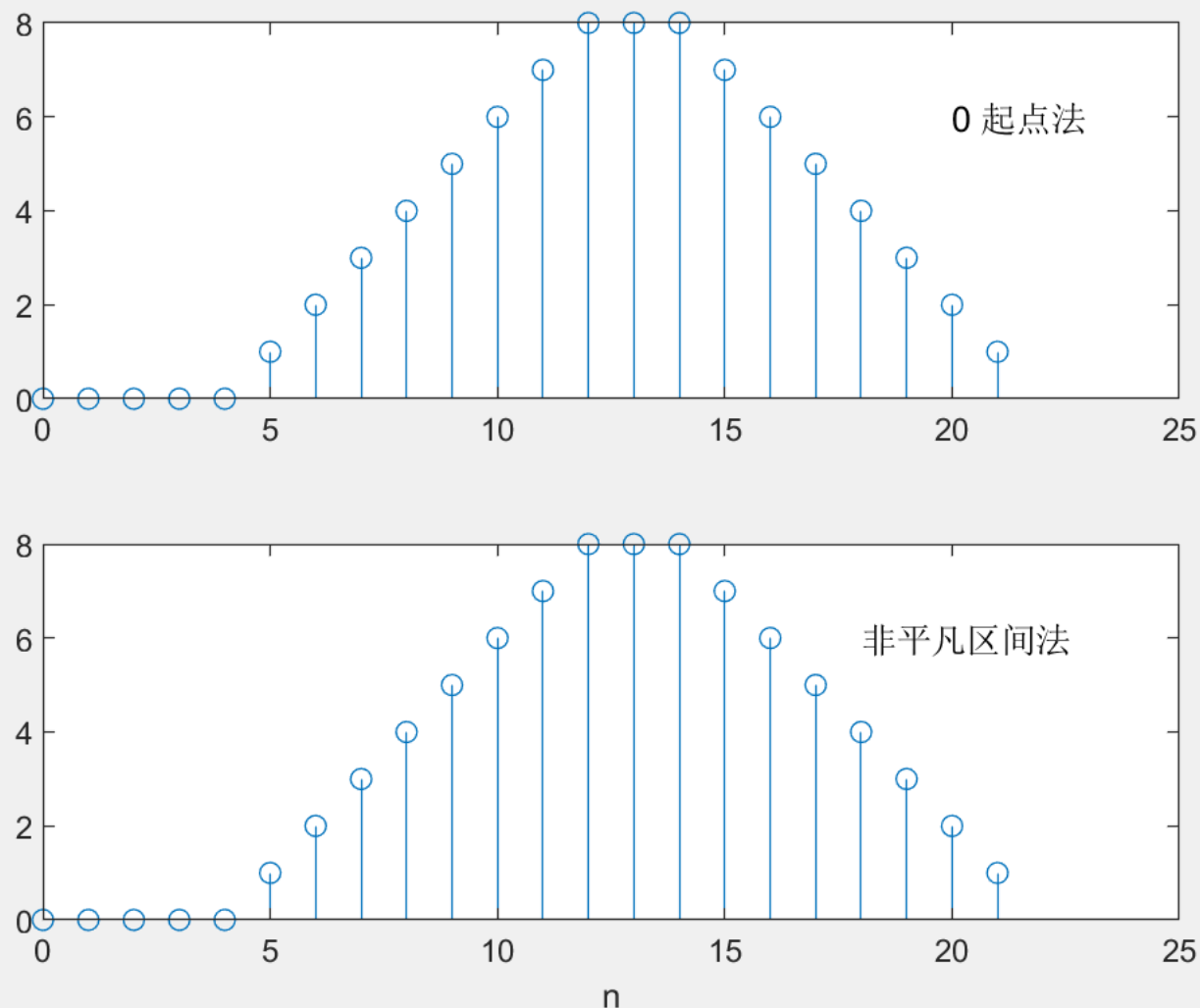
7 6

Columns 13 through 17

5 4 3 2 1

例4.4-7

```
subplot(2,1,1),stem(kc,c), text(20,6,'0 起点法')%方法二直接绘图  
CC=[zeros(1,KC(1)),C]; %方法三的结果左侧填充5个0  
subplot(2,1,2),stem(kc,CC),text(18,6,'非平凡区间法') %  
xlabel('n') %注：计算单点的卷积值使用sum函数速度更快
```



动态规划的引入-递归与递推

□ 再考虑斐波那契数列的定义：
$$a_n = \begin{cases} 1, & n = 1, 2 \\ a_{n-2} + a_{n-1}, & n \geq 3 \end{cases}$$

□ 递归式计算：利用子函数对自身的反复调用完成计算问题：

```
function r = fibo(a)
if(a>=3)
    r = fibo(a-2) + fibo(a-1);
else
    r = 1;
end
end
```

□ 运行时间表：

n	10	20	30	40	50
时间（秒）	0.000691	0.002367	0.061522	7.875290	766.792314

动态规划的引入-递归与递推

- ❑ 速度减慢的原因：大量的，不必要的重复的运算，使得算法复杂度达到 $O(2^n)$ 的级别。
- ❑ 递推式计算：避免重复计算，一气呵成。下列代码可以直接获得斐波那契数列前 n 项，复杂度 $O(n)$ ：

```
n = 1000;  
fibo = ones(1,n);  
for i = 3:n  
    fibo(i) = fibo(i-1) + fibo(i-2); %递推式，也称为状态转移方程  
end
```

- ❑ 运行时间仅需约0.0002秒。（应当大部分还是启动运行解释器的时间）。
- ❑ 当 n 特别大时，递归算法几近崩溃。而递推算法仍然可以在较短时间内完成计算。

动态规划例1-最长不降子序列问题

- 设有有限长度的数列 $a_1, a_2, \dots, a_n$ ，若存在子列 $a_{i_1}, a_{i_2}, \dots, a_{i_k}$ ，使得 $\forall m < n, a_{i_m} \leq a_{i_n}$ ，称子列为不降子序列。本问题的目的为计算 $a_1, a_2, \dots, a_n$ 的最长不降子序列长度。
- 如数列13, 7, 9, 5, 16, 38, 24, 37, 18, 44, 19, 21, 22, 63, 15。很容易获取一个不降子序列13, 16, 38, 44, 63，长度为5，但这个并不是最长不降子序列。但该数列的最长不降子序列为7, 9, 16, 18, 19, 21, 22, 63，长度为8。如果获取该子列呢？
- 暴力枚举法：（算法复杂度 $O(2^n)$ ，可以确保获得正确答案，但当 n 较大时，会出现过长的时间消耗，导致算法崩溃）
- 各种比较直接的快速算法（都不对）：如1. 从最小的数开始，此方法从5开始，只能得到长度为7的不降子序列。2. 分治法，以37为中点，左侧最优解7, 9, 16, 24, 37，右侧最优解18, 19, 21, 22, 63，总长度达到10，但两组解并不能衔接。

动态规划-保证正确的算法精简方式

- ❑ 动态规划的两大核心要件：**最优子结构**与**重叠子问题**。利用最优子结构的性质，减少暴力枚举中重复的运算与比较。
- ❑ 考虑数列 $a_1, a_2, \dots, a_k, a_{k+1}$ 的最长不降子序列问题。从最后一个元素出发，可以有两种选择。分析如下：
 - 1) 最长不降子序列不包含 a_{k+1} ，此时将 $a_1, a_2, \dots, a_k$ 的最长不降子序列结果援引过来即可。
 - 2) 最长不降子序列包含 a_{k+1} ，此时需确定其上一项 $a_j, 1 \leq j \leq n$ ，在满足前提 $a_j \leq a_{k+1}$ 的前提下，新序列长度可以等于 $(a_1, a_2, \dots, a_j$ 中包含 a_j 的最长不降子序列长度) + 1。
 - 3) 比较上述所有情况，获得 $a_1, a_2, \dots, a_k, a_{k+1}$ 的最长不降子序列的长度。
- ❑ 我们发现最关键的递推式比较依赖于 $a_1, a_2, \dots, a_j$ 之中包含 a_j 的最长不降子序列长度。我们记这个长度为 $F(j)$ 。

动态规划-保证正确的算法精简方式

- 对于本问题而言，最优子结构就是每一个 $F(j)$ 的值。因为计算 $F(k+1)$ 可以利用之前计算的 $F(1), F(2), \dots, F(k)$ 的结果，从而避免这些子问题的重复计算。
- 动态规划的递推式也叫作状态转移方程。对本问题而言，状态转移方程可以记为：

$$F(k+1) = \begin{cases} 1, & k = 0 \\ \max_{\substack{1 \leq j \leq k \\ a_j \leq a_{k+1}}} F(j) + 1, & k > 0 \end{cases}$$

此外，若后续 $F(k+1)$ 无法搜索到合适结果，也设 $F(k+1) = 1$ 。

- 因为最终获取的最长不降子序列未必包含最后一项，实际上的最长不降子序列长度应该等于 $\max_{1 \leq k \leq n} F(k)$ 。
- 因此，我们得到了一个算法复杂度为 $O(n^2)$ 的动态规划算法。

动态规划的算法代码

```
a = [13,7,9,5,16,38,24,37,18,44,19,21,22,63,15];  
n = length(a);  
F = ones(1,n);      %设立状态转移函数，代表包含末项的最长不降子序列长度  
for i = 2:n  
    temp = F(1,1:i-1);      %提取之前的状态转移函数  
    ind = a(1,1:i-1) <= a(i); %考虑符合不减条件的上一项  
    temp = ind.*temp;        %将不符合条件的转移函数暂乘为0  
    F(i) = max(temp) + 1;    %新的状态转移函数由此生成  
end  
Result = max(F)          %最大状态转移函数提供了问题的解
```

❑ 该代码可以得到最长子序列的长度，但无法直接获得子序列具体是什么，运行结果如下：

Result =

利用静态链表获得最长不降子序列 (了解)

```
a = [13,7,9,5,16,38,24,37,18,44,19,21,22,63,15];  
n = length(a);  
F = ones(1,n);      %设立状态转移函数，代表包含末项的最长不降子序列长度  
G = zeros(1,n);     %静态链表记录从1~i位置且包含ai的最长不降子序列  
  
for i = 2:n  
    temp = F(1,1:i-1);      %提取之前的状态转移函数  
    ind = a(1,1:i-1) <= a(i); %考虑符合不减条件的上一项  
    temp = ind.*temp;        %将不符合条件的转移函数暂乘为0  
  
    [maxv,maxp] = max(temp); %记录最长子串的位置  
    if (maxv~=0)             %如果有超过长度为1的最优子串  
        G(i) = maxp;         %静态链表记录最长不降子列上一个位置  
    end  
    F(i) = maxv + 1;          %新的状态转移函数由此生成  
end
```

利用静态链表获得最长不降子序列（了解）

```
[~,i] = max(F); %寻找最长不降子序列的最后一个位置
Result_ind = [i]; %初始化结果序列

while (G(i)~=0) %记录结果子列的下标
    Result_ind = [G(i),Result_ind];
    i = G(i);
end

Result = a(Result_ind) %显示结果子列
```

- 该代码可以直接输出一种符合最长不降子序列的序列结果，在本页最上面一行记录F的最大值，可以同时得到该长度：

Result =

7 9 16 18 19 21 22 63

动态规划例2-矩阵连乘问题

- 计算矩阵乘法 $A_{m \times n} B_{n \times p}$ ，一共需要得到 $m \times p$ 个元素，每个元素需要进行 n 次乘法和 $(n - 1)$ 次加法。因而算法消耗可以近似的用 mnp 来表示。
- 考虑实例 $A_{5 \times 1} B_{1 \times 5} C_{5 \times 1}$ 这样的连乘情况，先算 AB ，再算 ABC ，算法消耗总计为 $5 \times 1 \times 5 + 5 \times 5 \times 1 = 50$ 。而根据矩阵乘法的结合律，先算 BC ，再算 ABC 也可以得到正确答案，算法总消耗为 $1 \times 5 \times 1 + 5 \times 1 \times 1 = 10$ ，明显计算量更小。
- 现假设有矩阵 $A_1 A_2 \cdots A_n$ 这样的连乘问题，根据矩阵乘法的合理性，需要有行（列）数指标共 $(n + 1)$ 个，记为 $a_1, a_2, \dots, a_n$ ，求使用最佳顺序的情况下的最小算法消耗。
- 在暴力枚举的体系下，每一次都可以在任意两个矩阵的间隔中进行“抢先计算”，容易计算出枚举的情况共有 $(n - 1)!$ 种，算法复杂度 $O(n!)$ 比较高。

矩阵连乘问题的子结构分解与递推

- 更换思路，将中间某个位置的乘法看作最后一步，即 $(A_1A_2\cdots A_i)(A_{i+1}A_{i+2}\cdots A_n)$ 。在这种情况下，可以看成两更短的矩阵连乘问题，最后加上末步的计算代价 $a_1a_{i+1}a_n$ 。不过，由于 $1 \leq i \leq n-1$ ，我们需要考虑这 $(n-1)$ 种不同分割的结果，进行比较，得到最小的计算代价。
- 本问题的重复子问题与最优子结构就存在于 **一部分连续矩阵连乘的最小计算消耗**。定义 $F(i, j)$, $1 \leq i \leq j \leq n$ 为 $A_iA_{i+1}\cdots A_j$ 的最小计算消耗。我们可以得到递推式

$$F(i, j) = \begin{cases} 0, & j = i \\ \min_{i \leq k < j} F(i, k) + F(k+1, j) + a_i a_{k+1} a_{j+1}, & j > i \end{cases}$$

- 容易发现，上述算法通过2个矩阵连乘计算消耗，递推到3个矩阵连乘，4个矩阵连乘……，是一种金字塔式的递推结构。该算法的复杂度可以控制到 $O(n^3)$ ，远远超过暴力枚举。

动态规划的算法代码

```
a = [5,1,5,1,5,1,5,1];  
n = length(a)-1;  
F = zeros(n);           %状态转移方阵  
for len = 2:n           %连乘矩阵的个数  
    for i = 1:n-len+1    %连乘左端，此循环顺序至关重要  
        j = i + len - 1;  
        k = i:j-1;       %批量化运算，减少一层for循环  
        temp = F(i,k)+F(k+1,j)' + a(i)*a(k+1)*a(j+1);  
        %批量计算一组状态转移函数，注意向量类型要一致  
        F(i,j) = min(temp); %取最小值  
    end  
end  
Result = F(1,n)
```

Result =

动态规划例3-购物问题 (0-1背包问题)

- 假设某商店共有 n 件不同的商品，每件商品的价格是 c_i 元，且 $c_i \in \mathbb{N}_+$ ，每件商品的性能值为 v_i 元。现你手上共有 C 元 ($C \leq 2000$)，问如何购买可以使你购买物品的总性能值最大？
(每件物品都可以选择买或不买但都仅有1件, $v_i > 0$)
- 该问题的另一个版本称为背包问题，而且对每一件物品只有不买 (0) 和买 (1) 两种选择，因此该问题称为0-1背包问题。该问题的数学模型可以解释为一种整数规划问题：

$$\begin{aligned} \max_{\{w_i\} \in \{0,1\}} & \sum w_i v_i \\ \text{s. t. } & \sum w_i c_i \leq C \end{aligned}$$

- 暴力枚举所有情况，需要分别讨论每一件商品是否购买，并根据所有情况分析总价格和总性能值。算法复杂度达到 $O(2^n \cdot n)$ ， n 较大时，算法几近崩溃。根据“性价比”排序也同样存在错误。（高性价比物品有可能会致手上现金资源应用不充分，如 $\vec{c} = [7, 5, 4], \vec{v} = [8, 5, 4], C = 10$ ）

购物问题的最优子结构与递推

- 本题目最大的突破口在于 c_i, C 为整数且规模有限。因此利用不同的总价格下的最优购买总性能值构造状态转移方程，是本题的突破点。因此我们设状态函数 $F(i, j)$ 为使用不超过 j 元在前 i 件商品中选购时可以获得的最大性能值，其中 $0 \leq j \leq C$ 。
- 如果前 i 件物品用不超过 j 元购买的最优购买组合为 $\{k_1, k_2, \dots, k_l\}$ 且 $k_l = i$ （即第 i 件需购买），则类似于最长不降子序列问题可以发现，前 $(i-1)$ 件物品用不超过 $(j - c_i)$ 元购买的最优组合必包含 $\{k_1, k_2, \dots, k_{l-1}\}$ ，否则就会发生矛盾。
- 从而，当我们计算 $F(i, j)$ 并考虑第 i 件物品是否购买时，可以比较 $F(i-1, j - c_i) + v_i$ 与 $F(i-1, j)$ 的大小，对比该物品购买或不购买的最优情况进行“二选一”。状态转移公式为：

$$F(i, j) = \begin{cases} \max(F(i-1, j - c_i) + v_i, F(i-1, j)), & j \geq c_i \\ F(i-1, j), & j < c_i \end{cases}$$

上适用于 $i > 1$ ，而 $i = 1$ 的初始化比较简单，总复杂度 $O(nC)$ 。

动态规划的算法代码

```
c = [1,3,3,4,6,8,9,11,13];  
v = [1,2,3,5,6,10,9,10,15];  
C = 20;  
n = length(c);  
F = zeros(n,C+1); %因为j指标从0开始，需要多提供一列  
F(1,1:c(1)) = 0; %买不起第一件商品则无性能值  
F(1,c(1)+1:C+1) = v(1); %买得起第一件则获得第一件的性能值  
  
for i = 2:n  
    F(i,1:c(i)) = F(i-1,1:c(i)); %买不起第i件则无需比较  
    F(i,c(i)+1:C+1) = max(F(i-1,1:C-c(i)+1) + v(i), F(i-  
1,c(i)+1:C+1));  
    %当价格买得起时，买得起第i件则对比买与不买谁更优，取最大性能值  
end  
Result = F(n,C+1) %最终结果对应n件产品遍历后c元的最大性能值  
Result =
```

测试数据的设计（可用于自己实验）

```
c = [5,5,5,5,16]; v = [5,5,5,5,19]; C = 20; %高性价比不充分利用
```

```
Result = 20
```

```
c = [5,5,5,5,21]; v = [1,1,1,1,100]; C = 20; %高性价比无法利用
```

```
Result = 4
```

```
c = [5,10,5,2,4]; v = [1,8,1,6,2]; C = 20; %不充分利用带来最优解
```

```
Result = 16
```

```
c = [2      5      9      15      15      3      15      15      8      13];
```

```
v = [3      7      14      12      15      10      1      13      15      11];
```

```
C = 20; %随机生成数据样例（代码c=ceil(rand(1,10)*15)）
```

```
Result = 39（购买第3、6、9件商品，获得最优解需完成静态链表）
```

```
c = [12      12      6      10      3      11      1      5      1      2];
```

```
v = [13      11      5      15      1      7      6      12      12      3];
```

```
C = 20; %随机生成数据样例
```

```
Result = 48（购买第4、7、8、9、10件商品）
```

第十五周电子版作业（下周二晚以前提交）

- ❑ 作业请于12月14日（周二）23:59:59或以前提交到课堂派平台。（请同学们加入正确的班级）
- ❑ 作业文件名格式：“05370038；李嘉；第十五周作业”
- ❑ 请将所有题目的方法设计（如果有）、代码、运行结果、绘图结果（如果有），放于同一个pdf（推荐使用）文件内（doc或docx也可）。文件名建议与邮件标题完全相同。
- ❑ 作业满分52分，电子版作业迟交者会按照每天5分的梯度损失分数。分数会根据答案的正确与否、算法的高效与否、文档展示的规范性等多个方面综合评定。
- ❑ 请勿抄袭，一经发现，最终总评不超过80。两次抄袭等于挂科。如果进行了部分方法与内容的借鉴，请在作业明示你的参考文献或借鉴人。

第十五周电子版作业

□ Q1. 习题4第17题 (写出MATLAB源代码与运行结果)

已知有理分式 $R(x) = \frac{N(x)}{D(x)}$ ，其中 $N(x) = (3x^3 + x)(x^3 + 0.5)$ ， $D(x) = (x^2 + 2x - 2)(5x^3 + 2x^2 + 1)$ 。(1) 求

该分式的商多项式 $Q(x)$ 和余多项式 $r(x)$ 。(2) 用程序验算 $D(x)Q(x) + r(x) = N(x)$ 是否成立。(提示：采用范数指令 `norm` 验算。)

□ Q2. 设信号 $x(t) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{t^2}{2\sigma^2}}$, $-2 \leq t \leq 2$, $y(t) = \begin{cases} 1, & |t| \leq 1 \\ 0, & |t| > 1 \end{cases}$

为矩形脉冲函数，现用数值方法模拟两个具有紧支集函数的卷积结果，采样步长设置 1×10^{-4} ，而参数 $\sigma = 0.1$ 。写出你定义“离散化”的数字信号以及计算离散卷积 $x(t) * y(t)$ 的代码，并提供绘图结果。根据你的计算或绘图结果，一个光滑信号 $x(t)$ 与粗糙信号 $y(t)$ 的卷积结果 $x(t) * y(t)$ 是光滑（高阶可导）的还是粗糙的？ $\sigma \rightarrow 0$ 时， $x(t) * y(t)$ 会有什么变化，请尝试解释之。

第十五周电子版作业

- Q3. 购物问题的升级：仍然假设某商店共有 n 件不同的商品，其中前 m 件物品为特殊物品， $1 < m < n$ 。每件商品的价格是 c_i 元，且 $c_i \in \mathbb{N}_+$ ，每件商品的性能值为 v_i 元。现你手上共有 C 元（ $C \leq 2000$ ），并要求至少购买一件特殊物品，问如何购买可以使你购买物品的总性能值最大？（数据保证至少有一组解，每件物品都可以选择买或不买但都仅有1件， $v_i > 0$ ）
- 读取数据文件Q31.mat, Q32.mat, Q33.mat, Q34.mat可以分别获得四组不同的数据。每一组分别包含常数 m, n, C ，价格与性能向量 $\vec{c}, \vec{v}$ 。
- 请将你的算法解释、代码、运行结果包含在作业之中。本题仅
需计算出最佳购买方案的总性能值，无需提供具体购买方案。
建议使用fprintf命令对结果进行展示。

课后反馈（不用交）

- 将今天讲过的例题尝试自己键入并运行一遍
- 熟悉离散卷积函数`conv`的概念，以及三种边界条件，为下一周的反卷积相关内容做知识的储备。
- 思考动态规划的原理，对本堂课介绍的三类问题“最长不降子序列问题”、“矩阵连乘问题”、“0-1背包问题”的问题和方法有较好的理解。

感谢同学们认真听课!

欢迎同学们积极提问、交流

mcstj@mail.sysu.edu.cn